SAVITRIBAI PHULE PUNE UNIVERSITY

A MINI PROJECT REPORT ON

Automated Testing of an E-Commerce Website Using Selenium WebDriver

SUBMITTED TOWARDS THE
PARTIAL FULFILLMENT OF THE REQUIREMENTS OF
LABORATORY PRACTICE IV (SOFTWARE TESTING AND QUALITY ASSURANCE)

BACHELOR OF ENGINEERING (Computer Engineering)

BY

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

Under The Guidance of

Prof. Guide Name

Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING
NBN SINHGAD SCHOOL OF ENGINEERING
10/1, AMBEGAON (BK), PUNE-411041

2026-27

Sinhgad Institutes

NBN SINHGAD SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the Mini Project Entitled

Automated Testing of an E-Commerce Website

Submitted by

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

is a bonafide work carried out by Students under the supervision of Prof. Guide Name and it is submitted towards the partial fulfillment of the requirement of Bachelor of Engineering (Computer Engineering) for the course Laboratory Practice IV.

Prof. Guide Name
Internal Guide
Dept. of Computer Engg.

Dr. S. P. Bendale
H.O.D
Dept. of Computer Engg.

Dr. S. P. Patil
Principal
NBN Sinhgad School of Engineering

Signature of Internal Examiner

Signature of External Examiner

PROJECT APPROVAL SHEET

Automated Testing of an E-Commerce Website Using Selenium WebDriver

Is successfully completed by

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

at

DEPARTMENT OF COMPUTER ENGINEERING

NBN SINHGAD SCHOOL OF ENGINEERING, PUNE

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

ACADEMIC YEAR 2026-2027

Prof. Guide Name	Dr. S. P. Bendale
Internal Guide	H.O.D
Dept. of Computer Engg.	Dept. of Computer Engg.

Abstract

Every change to a web application risks breaking features that already worked. Running the same manual test cases again after every change is slow and error-prone, which is why stable, repeatable test cases are automated. This mini project builds an automated regression test suite for **Swag Labs** (`https://www.saucedemo.com`), a public demo e-commerce website with login, product catalogue, shopping cart and checkout, using **Selenium WebDriver** driven from **Python** with the **pytest** framework.

The passing test cases from the companion manual-testing project were selected for automation. The suite is organised with a light Page Object Model: four small classes (`LoginPage`, `InventoryPage`, `CartPage`, `CheckoutPage`) hold the element locators and user actions of each page, so that tests read like user stories and locator changes are confined to one file. A pytest fixture starts a fresh headless Microsoft Edge browser for every test and closes it afterwards; a hook captures a screenshot whenever a test fails and embeds it in a self-contained HTML report produced by pytest-html. Parametrisation is used to run the same login and checkout validation logic with several data sets, and a known defect of the `problem_user` account is kept in the suite as an expected failure so that the day it is fixed the suite will report it.

The final suite contains 22 test functions across five modules (login, products, cart, checkout, navigation). A full run in headless Edge takes about 105 seconds and the latest execution reports 21 passed and 1 expected failure. The project demonstrates locator strategies (ID, class name, CSS selector), explicit waits, test fixtures, data-driven tests, reporting and failure diagnosis, which are the core skills of Selenium-based test automation taught in the Software Testing and Quality Assurance course.

Acknowledgments

It gives us great pleasure in presenting the mini project report on ***'Automated Testing of an E-Commerce Website'****.*

We would like to take this opportunity to thank our internal guide ***Prof. Guide Name*** *for giving us all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

We are also grateful to ***Dr. S. P. Bendale****, Head of Computer Engineering Department, NBN Sinhgad School of Engineering, for his indispensable support and suggestions.*

In the end our special thanks to the ***laboratory staff of the Department of Computer Engineering*** *for providing various resources such as the laboratory with all needed software platforms and continuous Internet connection for our project.*

Vishwajeet Tulse
Student Name 2
Student Name 3
Student Name 4
(B.E. Computer Engg.)

Contents

1 Synopsis **1**
- 1.1 Project Title . . . 2
- 1.2 Project Option . . . 2
- 1.3 Internal Guide . . . 2
- 1.4 Sponsorship and External Guide . . . 2
- 1.5 Technical Keywords (As per ACM Keywords) . . . 2
- 1.6 Problem Statement . . . 2
- 1.7 Abstract . . . 2
- 1.8 Goals and Objectives . . . 3
- 1.9 Relevant mathematics associated with the Project . . . 3
- 1.10 Names of Conferences / Journals where papers can be published . . . 4
- 1.11 Review of Conference/Journal Papers supporting Project idea . . . 4
- 1.12 Plan of Project Execution . . . 4

2 Technical Keywords **5**
- 2.1 Area of Project . . . 6
- 2.2 Technical Keywords . . . 6

3 Introduction **7**
- 3.1 Project Idea . . . 8
- 3.2 Motivation of the Project . . . 8
- 3.3 Literature Survey . . . 8

4 Problem Definition and scope **10**
- 4.1 Problem Statement . . . 11
  - 4.1.1 Goals and objectives . . . 11
  - 4.1.2 Statement of scope . . . 11
- 4.2 Major Constraints . . . 11
- 4.3 Methodologies of Problem solving and efficiency issues . . . 11
- 4.4 Outcome . . . 12
- 4.5 Applications . . . 12
- 4.6 Hardware Resources Required . . . 12
- 4.7 Software Resources Required . . . 12

5 Project Plan **13**
- 5.1 Project Estimates . . . 14
  - 5.1.1 Reconciled Estimates . . . 14
  - 5.1.2 Project Resources . . . 14
- 5.2 Risk Management w.r.t. NP Hard analysis . . . 14

5.2.1 Risk Identification . . . 14
5.2.2 Risk Analysis . . . 17
5.2.3 Overview of Risk Mitigation, Monitoring, Management . . . 17
5.3 Project Schedule . . . 17
5.3.1 Project task set . . . 17
5.3.2 Task network . . . 17
5.3.3 Timeline Chart . . . 18
5.4 Team Organization . . . 18
5.4.1 Team structure . . . 18
5.4.2 Management reporting and communication . . . 18

6 Software requirement specification 19
6.1 Introduction . . . 20
6.1.1 Purpose and Scope of Document . . . 20
6.1.2 Overview of responsibilities of Developer . . . 20
6.2 Usage Scenario . . . 20
6.2.1 User profiles . . . 20
6.2.2 Use-cases . . . 21
6.2.3 Use Case View . . . 22
6.3 Data Model and Description . . . 22
6.3.1 Data Description . . . 22
6.3.2 Data objects and Relationships . . . 22
6.4 Functional Model and Description . . . 23
6.4.1 Data Flow Diagram . . . 23
6.4.2 Activity Diagram: . . . 24
6.4.3 Non Functional Requirements: . . . 24
6.4.4 State Diagram: . . . 25
6.4.5 Design Constraints . . . 25
6.4.6 Software Interface Description . . . 25

7 Detailed Design Document using Appendix A and B 26
7.1 Introduction . . . 27
7.2 Architectural Design . . . 27
7.3 Data design (using Appendices A and B) . . . 27
7.3.1 Internal software data structure . . . 27
7.3.2 Global data structure . . . 27
7.3.3 Temporary data structure . . . 27
7.3.4 Database description . . . 28
7.4 Component Design . . . 28
7.4.1 Class Diagram . . . 28

8 Project Implementation 29
8.1 Introduction . . . 30
8.2 Tools and Technologies Used . . . 30
8.3 Methodologies/Algorithm Details . . . 30
8.3.1 Algorithm 1/Pseudo Code . . . 30
8.3.2 Algorithm 2/Pseudo Code . . . 30
8.4 Verification and Validation for Acceptance . . . 31

9 Software Testing **32**
9.1 Type of Testing Used 33
9.2 Test Cases and Test Results 34

10 Results **36**
10.1 Screen shots 37
10.2 Outputs 38

11 Deployment and Maintenance **39**
11.1 Installation and un-installation 40
11.2 User help 40

12 Conclusion and Future Scope **41**

13 References **43**

Annexure A Laboratory assignments on Project Analysis of Algorithmic Design **45**

Annexure B Laboratory assignments on Project Quality and Reliability Testing of Project Design **47**

Annexure C Project Planner **49**

Annexure D Reviewers Comments of Paper Submitted **51**

Annexure E Plagiarism Report **53**

Annexure F Term-II Project Laboratory Assignments **55**

Annexure G Information of Project Group Members **57**

List of Figures

5.1 Project timeline (Gantt chart) 18

6.1 Use case diagram 22
6.2 Entity relationship of suite data 22
6.3 Level 0 DFD 23
6.4 Level 1 DFD 23
6.5 Activity diagram of a test run 24
6.6 State transition diagram of a test 25

7.1 Architecture of the automation framework 27
7.2 Class diagram of the page objects 28

10.1 pytest-html report of the final run 37
10.2 Screenshot captured automatically when the expected-failure test is forced to run: prices are not sorted for problem_user 38

List of Tables

4.1 Hardware Requirements 12

5.1 Time estimates 14
5.2 Risk Table 15
5.3 Risk Probability definitions [11] 15
5.4 Risk Impact definitions [11] 15

6.1 Use Cases 21

9.1 Automated test cases and results (final run: 21 passed, 1 xfailed, 105.6 s) 34

C.1 Project planner 50

CHAPTER 1
SYNOPSIS

1.1 Project Title

Automated Testing of an E-Commerce Website Using Selenium WebDriver

1.2 Project Option

Internal project (mini project for Laboratory Practice IV, Software Testing and Quality Assurance).

1.3 Internal Guide

Prof. Guide Name

1.4 Sponsorship and External Guide

Not applicable. The project is not sponsored.

1.5 Technical Keywords (As per ACM Keywords)

1. Software and its engineering
   (a) Software verification and validation
      i. Software testing and debugging
      ii. Regression testing, test automation
   (b) Software notations and tools
      i. Scripting languages (Python), testing frameworks
2. Information systems
   (a) Web applications, electronic commerce

1.6 Problem Statement

Design and implement an automated functional regression test suite for an e-commerce website (Swag Labs) using Selenium WebDriver, covering login, product listing and sorting, shopping cart, checkout and navigation, with reusable page objects, data-driven test cases, automatic screenshots on failure and an HTML execution report.

1.7 Abstract

- Manual regression testing of a web shop after every release is slow and repetitive; automation makes it fast and repeatable.

- Selenium WebDriver drives a real browser through the W3C WebDriver protocol, so tests exercise the application exactly as a user would.
- The suite uses Python, pytest and a Page Object Model; 22 tests cover five modules.
- Fixtures give every test a fresh browser; a hook captures screenshots on failure; pytest-html produces a single report file.
- The latest run: 21 passed, 1 expected failure (a documented site defect), in about 105 seconds.

1.8 Goals and Objectives

- To set up Selenium WebDriver with Python and a browser driver on Windows.
- To automate the stable test cases of the Swag Labs purchase flow.
- To apply the Page Object Model so that tests are readable and maintainable.
- To use pytest fixtures, parametrisation and markers (xfail) effectively.
- To generate an HTML report with screenshots of failures.
- To keep the code small and easy to explain.

1.9 Relevant mathematics associated with the Project

System Description:

- **Input:** test suite $S = \{t_1, \ldots, t_{22}\}$; browser b; the live application A; test data sets D_i for parametrised tests.
- **Output:** verdict $v : S \to \{\text{pass}, \text{fail}, \text{xfail}\}$; HTML report; screenshot set for failed tests.
- **Data structures:** locator tuples $(by, value)$; page object classes; pytest fixture graph; lists of product names and prices used in sort assertions.
- **Functions:**
  - $setup : b \to driver$ (fixture), $teardown : driver \to \emptyset$.
  - $action : (page, input) \to page'$ (click, type, select).
  - $assert$: $observed = expected$; e.g. for sorting, $prices = \text{sort}(prices)$; for checkout, $total = subtotal + \text{round}(0.08 \cdot subtotal, 2)$.
- **Mathematical formulation:** Each test is a predicate $P_i(A)$; the suite passes iff $\bigwedge_i P_i(A)$ holds for all tests not marked xfail. Running time $T = \sum_i (t_{setup} + t_{actions,i})$, about 105 s for 22 tests, versus roughly 2 hours for the same cases by hand.
- **Success Conditions:** browser driver starts; application reachable; all assertions hold; report generated.
- **Failure Conditions:** element not found (locator changed), timeout (slow page), assertion failure (defect); each produces a screenshot.

1.10 Names of Conferences / Journals where papers can be published

- IEEE International Conference on Software Testing, Verification and Validation (ICST)
- ACM International Symposium on Software Testing and Analysis (ISSTA)
- International Workshop on Automation of Software Test (AST)
- SPPU-sponsored and IIT-hosted student technical conferences / workshops

1.11 Review of Conference/Journal Papers supporting Project idea

1. Garousi and Mäntylä [1] review when test automation pays off: stable, repeated, regression-type tests, which is exactly our selection criterion.
2. Leotta et al. [2, 3] measure the maintenance benefit of the Page Object pattern and compare locator strategies, motivating our design.
3. Bruns, Kornstadt and Wichmann [4] describe web application testing with Selenium in practice.
4. Ramya, Sindhura and Sagar [5] compare automation tools and recommend Selenium for web GUIs.
5. Rafi et al. [6] summarise benefits and limitations of automation found in a systematic review.
6. Berner, Weber and Keller [7] give lessons learned on keeping automated suites maintainable.
7. The Selenium [8] and pytest [9] documentation define the APIs used.
8. Pressman [11] supplies the project-planning and risk framework.

1.12 Plan of Project Execution

Four weeks: (1) environment setup and study of Selenium/pytest, (2) page objects and login/products tests, (3) cart, checkout and navigation tests, reporting and screenshots, (4) full runs, repository and report. See Annexure C.

CHAPTER 2
TECHNICAL KEYWORDS

2.1 Area of Project

Software Testing and Quality Assurance: automated functional and regression testing of web applications.

2.2 Technical Keywords

1. Software and its engineering
   (a) Software testing and debugging; regression testing; test automation
   (b) Software design patterns (Page Object Model)
2. Information systems
   (a) Web applications; browsers; electronic commerce
3. Computing methodologies
   (a) Scripting languages (Python)

CHAPTER 3
INTRODUCTION

3.1 Project Idea

- Convert the stable test cases of a web shop into Selenium scripts that a computer can run in minutes, every time the application changes.
- Keep the design simple: one Page Object class per page, one test file per module, one fixture that opens and closes the browser, one command to run everything and get a report.
- Use the public Swag Labs demo so that no server setup is needed and the suite can be run anywhere.

3.2 Motivation of the Project

- Regression suites are executed many times; automation turns a two-hour manual run into a two-minute unattended run.
- Selenium WebDriver is the industry-standard open-source tool for browser automation and is part of the LP-IV syllabus (Assignment 10).
- The companion manual-testing project produced a set of well-defined test cases with known expected results, which is the ideal input for automation.

3.3 Literature Survey

- **Garousi and Mäntylä (2016)** [1]: multi-vocal review on when and what to automate; recommends automating stable regression tests and keeping exploratory tests manual. Our suite automates only cases that passed manually.
- **Leotta et al. (2013)** [2]: industrial case study showing that Page Object based Selenium suites need far less maintenance effort when the application changes than scripts with inline locators.
- **Leotta et al. (2014)** [3]: compares ID-based and XPath locators; ID locators are more robust, which is why the suite uses IDs wherever the application provides them.
- **Bruns, Kornstadt and Wichmann (2009)** [4]: early experience report on Selenium for web application testing, including the need for waits and stable test data.
- **Ramya, Sindhura and Sagar (2017)** [5]: survey of automation tools (Selenium, QTP, TestComplete) concluding Selenium is the most suitable free tool for web applications.
- **Rafi et al. (2012)** [6]: systematic literature review on benefits (reusability, repeatability, cost over time) and limitations (initial effort, maintenance) of test automation.

- **Berner, Weber and Keller (2005)** [7]: lessons from industrial automation projects: keep tests independent, make failures diagnosable (our screenshot hook), run often.
- **Selenium documentation** [8]: WebDriver API, locator strategies, waits, Page Object pattern.
- **pytest documentation** [9]: fixtures, parametrisation, markers, hooks; pytest-html plugin for reports.
- **Sauce Labs** [10]: description of the Swag Labs demo and its test users.

CHAPTER 4
PROBLEM DEFINITION AND SCOPE

4.1 Problem Statement

Build an automated test suite that logs in to Swag Labs, verifies the product list and sorting, adds and removes items in the cart, completes checkout with correct totals, checks form validation and navigation, and reports results with evidence, so that the whole regression run needs no human interaction.

4.1.1 Goals and objectives

- Input: the application URL, test user credentials, product identifiers and test data embedded in the tests; command-line options for browser and headed/headless mode.
- Output: console verdicts, `reports/report.html`, screenshots of failed tests.
- Objective: full, unattended regression run under three minutes with clear failure diagnostics.

4.1.2 Statement of scope

- In scope: functional tests of login (positive and negative), product listing and sorting, cart operations, checkout validation and totals, order completion, logout and protected-page access, in Microsoft Edge (Chrome supported by option).
- Out of scope: visual/pixel comparison, performance measurement, mobile browsers, tests of deliberately broken users except one documented expected failure.

4.2 Major Constraints

- The application is public and may change its DOM; locators are centralised in page objects to limit the impact.
- Tests must be independent: each test gets a fresh browser session, so no test depends on another's cart state.
- Network latency varies; explicit and implicit waits are used instead of fixed sleeps.
- The suite must run headless on a laboratory machine without a display and without Java (Python bindings are used).

4.3 Methodologies of Problem solving and efficiency issues

- **Record and playback (Selenium IDE):** quick but produces brittle scripts with duplicated locators.
- **Linear scripts with inline locators:** simple but every DOM change requires editing many tests.

- **Page Object Model with pytest (chosen):** locators and actions live in one class per page; tests are short and readable; fixtures remove setup duplication. Cost: one extra file. Efficiency: a fresh browser per test adds about 2 seconds each but guarantees isolation; the full suite runs in about 105 seconds.

4.4 Outcome

- A working Selenium/pytest suite of 22 tests with page objects, fixtures, parametrisation and an xfail marker.
- Self-contained HTML report and screenshot-on-failure mechanism.
- GitHub repository with documentation and this report.

4.5 Applications

- Nightly or per-commit regression testing of e-commerce sites in a CI pipeline.
- Smoke testing after deployment to production.
- Template for automating any form-based web application.

4.6 Hardware Resources Required

Sr. No.	Parameter	Minimum Requirement	Justification
1	CPU Speed	2 GHz dual core	A headless browser instance per test
2	RAM	4 GB	Browser plus Python interpreter
3	Network	Broadband Internet	Application is hosted online

Table 4.1: Hardware Requirements

4.7 Software Resources Required

Platform :

1. Operating System: Windows 11 (Linux/macOS also supported)
2. IDE: Visual Studio Code
3. Programming Language: Python 3.14 with Selenium 4.51, pytest 9, pytest-html 4
4. Browser: Microsoft Edge 142 (driver managed automatically by Selenium Manager)
5. Version control: Git and GitHub

CHAPTER 5
PROJECT PLAN

5.1 Project Estimates

A waterfall model was followed: requirements (select cases), design (page objects), implementation (tests), testing (runs), documentation.

5.1.1 Reconciled Estimates

5.1.1.1 Cost Estimate

All tools are free and open source. Effort about 40 person-hours for four students.

5.1.1.2 Time Estimates

Phase	Duration
Environment setup, Selenium and pytest study	3 days
Page object design, login and products tests	4 days
Cart, checkout, navigation tests; report and screenshot hook	5 days
Full runs, debugging, code review	2 days
Report and repository	4 days
Total	**18 working days**

Table 5.1: Time estimates

5.1.2 Project Resources

People: four students and one guide. Hardware: one laptop per student. Software: Python, Selenium, pytest, Edge, VS Code, Git/GitHub, MiKTeX.

5.2 Risk Management w.r.t. NP Hard analysis

Test execution is linear in the number of tests; no NP-hard component exists. Practical risks follow.

5.2.1 Risk Identification

Risks were identified from the questionnaire in [11]: requirements (the selected manual cases) are stable; the team has Python skills; the main uncertainty is the external website and browser/driver compatibility.

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Application DOM or behaviour changes, breaking locators	Medium	Medium	High	Medium
2	Flaky tests due to timing (page not yet loaded)	High	Medium	High	High
3	Browser/driver version mismatch on lab machines	Medium	Medium	Low	Medium

Table 5.2: Risk Table

Probability	Value	Description
High	Probability of occurrence is	$> 75\%$
Medium	Probability of occurrence is	$26 - 75\%$
Low	Probability of occurrence is	$< 25\%$

Table 5.3: Risk Probability definitions [11]

Impact	Value	Description
Very high	$> 10\%$	Schedule impact or Unacceptable quality
High	$5 - 10\%$	Schedule impact or Some parts of the project have low quality
Medium	$< 5\%$	Schedule impact or Barely noticeable degradation in quality

Table 5.4: Risk Impact definitions [11]

Risk ID	1
Risk Description	Application DOM or behaviour changes
Category	Technology / external dependency
Source	Public demo site maintained by a third party
Probability	Medium
Impact	High
Response	Mitigate
Strategy	All locators in `pages.py`; prefer stable ID locators; screenshot on failure shows what changed
Risk Status	Mitigated

Risk ID	2
Risk Description	Flaky tests due to timing
Category	Technology
Source	Observed in the first full run: login assertion checked the URL before navigation finished
Probability	High
Impact	High
Response	Mitigate
Strategy	Implicit wait of 5 s for elements plus an explicit `WebDriverWait` after login that waits for either the Products URL or an error message
Risk Status	Occurred and resolved

Risk ID	3
Risk Description	Browser/driver version mismatch
Category	Development Environment
Source	Different lab machines
Probability	Medium
Impact	Low
Response	Mitigate
Strategy	Selenium 4 Manager downloads the matching driver automatically; `--browser chrome` option as fallback
Risk Status	Mitigated

5.2.2 Risk Analysis

5.2.3 Overview of Risk Mitigation, Monitoring, Management

5.3 Project Schedule

5.3.1 Project task set

- Task 1: Install Python, Selenium, pytest; verify a first script opens the site.
- Task 2: Design page objects and the driver fixture; automate login and products tests.
- Task 3: Automate cart, checkout and navigation tests; add parametrisation and xfail.
- Task 4: Add HTML report and screenshot-on-failure; run the full suite and fix flakiness.
- Task 5: Repository, README and project report.

5.3.2 Task network

Task 1 → Task 2 → Task 3 → Task 4 → Task 5; Task 4 reporting work was started in parallel with Task 3.

5.3.3 Timeline Chart

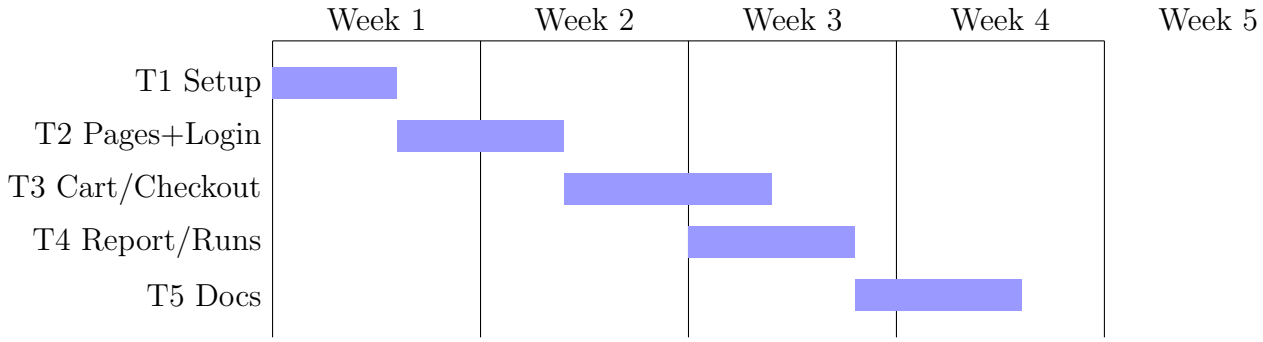


Figure 5.1: Project timeline (Gantt chart)

Please refer to Annexure C for the planner.

5.4 Team Organization

5.4.1 Team structure

Four students: one owns the framework (`conftest.py`, `pages.py`, reporting), one the login and products tests, one the cart and checkout tests, one the navigation tests, documentation and runs. Code was reviewed through GitHub commits.

5.4.2 Management reporting and communication

Weekly demonstration of the suite to the guide during the LP-IV slot; shared GitHub repository; group chat for coordination.

CHAPTER 6

SOFTWARE REQUIREMENT SPECIFICATION

6.1 Introduction

6.1.1 Purpose and Scope of Document

This SRS states what the automated test suite must do: which features of Swag Labs it verifies, how it is run and what it produces.

6.1.2 Overview of responsibilities of Developer

Select test cases, design page objects, implement tests and fixtures, configure reporting, run and stabilise the suite, document usage.

6.2 Usage Scenario

6.2.1 User profiles

- **Test engineer:** runs the suite from the command line, reads the HTML report, maintains page objects.
- **Developer:** runs the suite after a change to confirm nothing broke.
- **CI server (future):** runs the suite automatically on every commit.

6.2.2 Use-cases

Sr No.	Use Case	Description	Actors	Assumptions
1	Run suite	Execute all tests headless and produce the HTML report	Test engineer	Python, Selenium and Edge installed
2	Run subset	Execute one module or one test by name	Test engineer	–
3	Run headed	Watch the browser while tests run	Test engineer	Display available
4	Analyse failure	Open the report and the failure screenshot	Test engineer	A test failed
5	Maintain locators	Update `pages.py` after a DOM change	Test engineer	Site changed
6	Add test	Write a new test function using page objects	Test engineer	Page object exists

Table 6.1: Use Cases

6.2.3 Use Case View

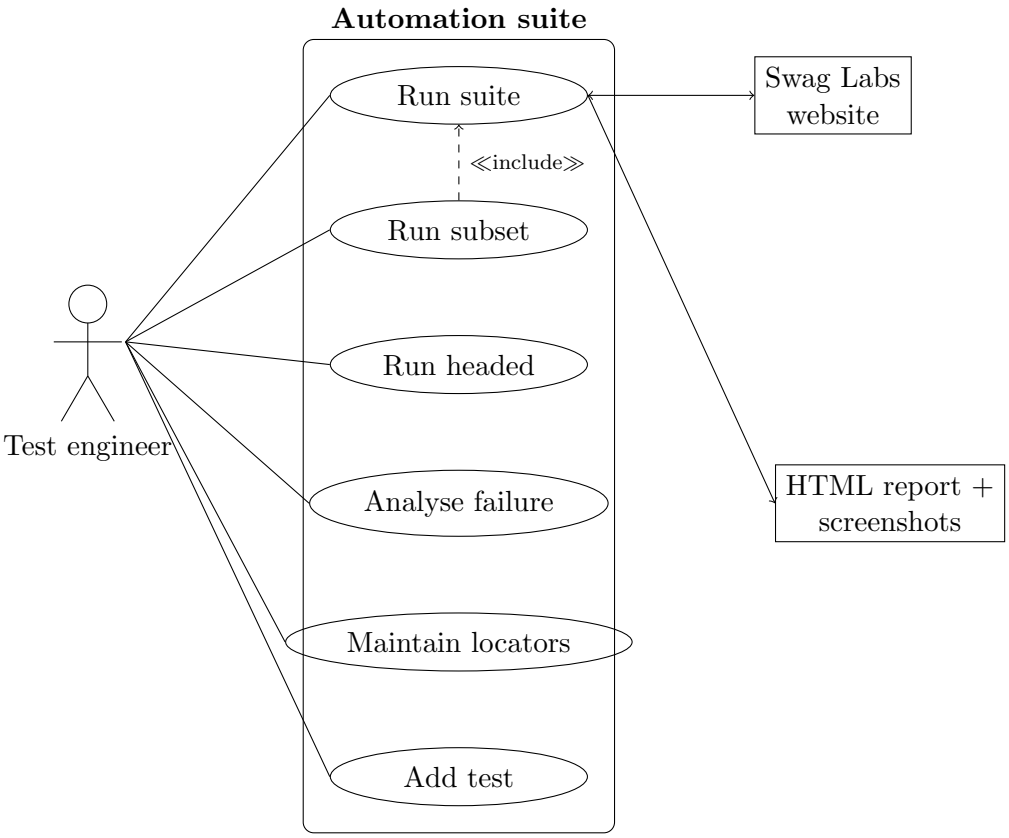


Figure 6.1: Use case diagram

6.3 Data Model and Description

6.3.1 Data Description

The suite holds three kinds of data: *locators* (strategy, value) in the page objects; *test data* (usernames, passwords, names, postal codes, expected messages) in parametrised tests; *results* (verdict, duration, screenshot path) collected by pytest.

6.3.2 Data objects and Relationships

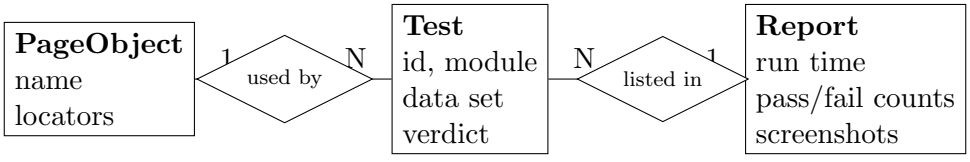


Figure 6.2: Entity relationship of suite data

6.4 Functional Model and Description

6.4.1 Data Flow Diagram

6.4.1.1 Level 0 Data Flow Diagram

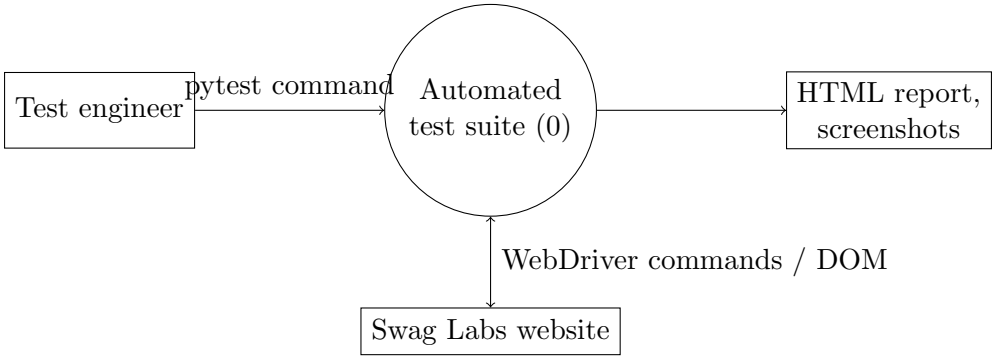


Figure 6.3: Level 0 DFD

6.4.1.2 Level 1 Data Flow Diagram

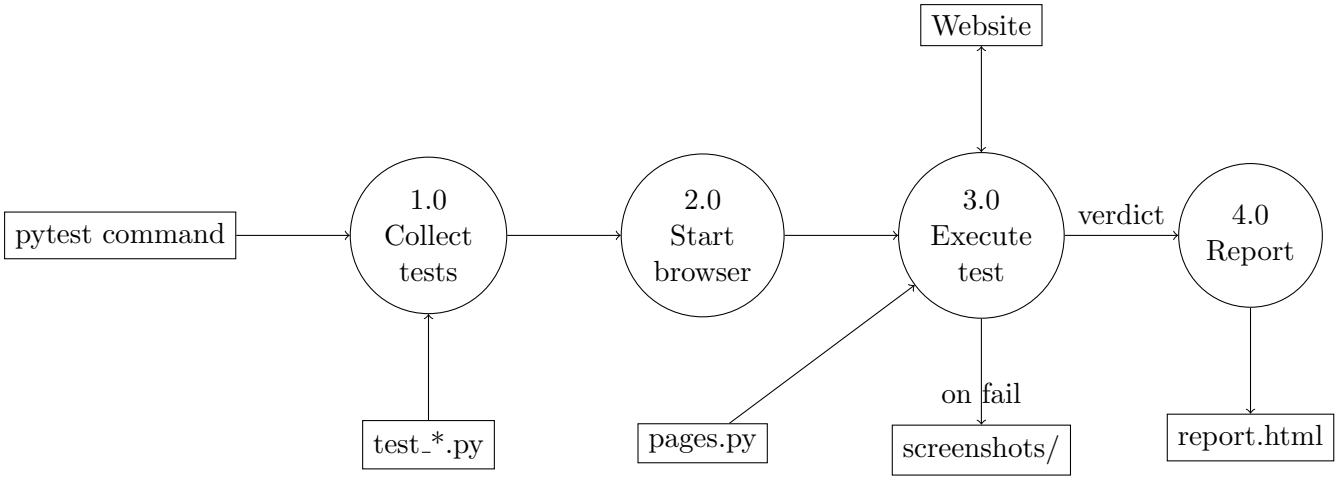


Figure 6.4: Level 1 DFD

6.4.2 Activity Diagram:

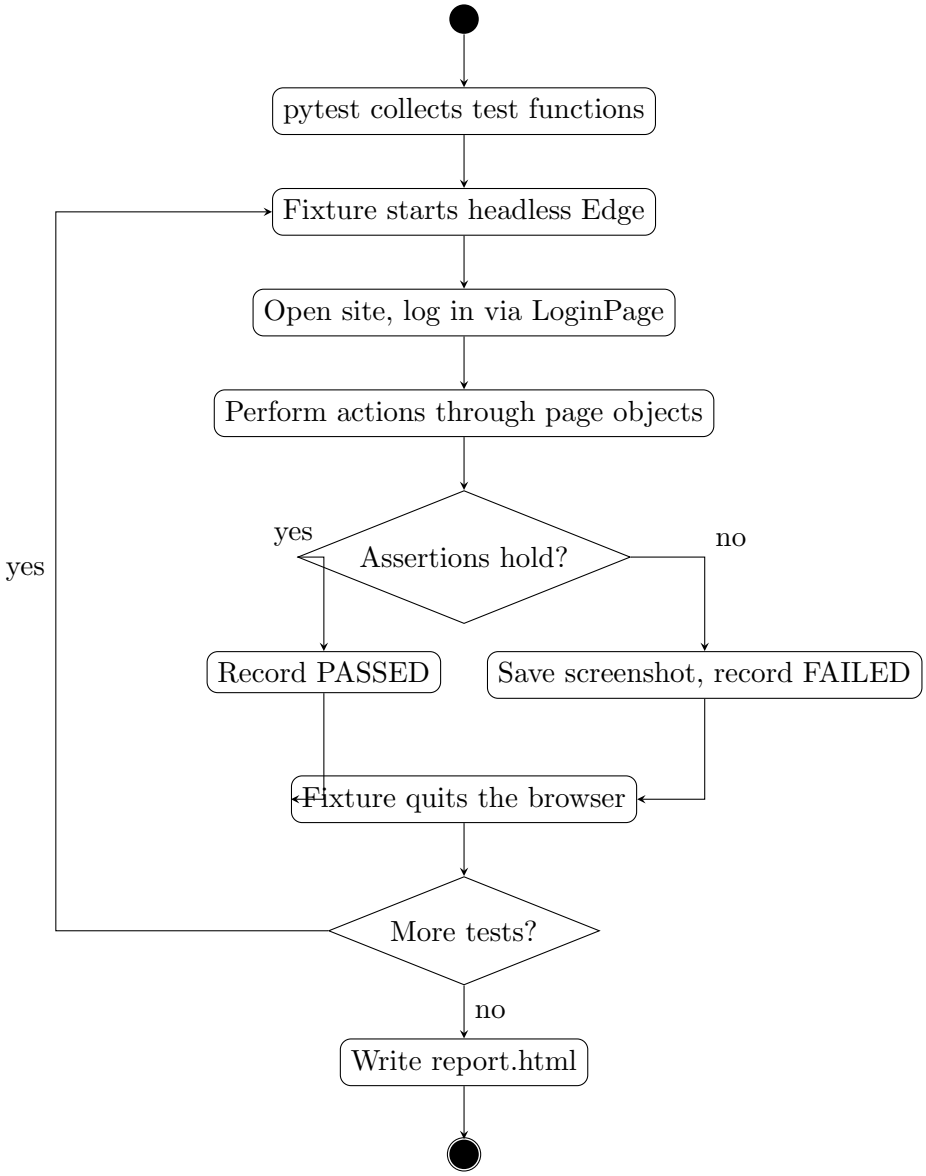


Figure 6.5: Activity diagram of a test run

6.4.3 Non Functional Requirements:

- **Interface Requirements:** command line (`python -m pytest`) with options `--browser` and `--headed`; HTML report viewable in any browser.
- **Performance Requirements:** full suite under 3 minutes; currently about 105 s.
- **Reliability:** tests independent and repeatable; waits instead of sleeps.
- **Modifiability:** locators only in `pages.py`; tests contain no locators.
- **Portability:** Python and Selenium run on Windows, Linux and macOS; Edge or Chrome.

- **Testability / Usability:** one command runs everything; failures include a screenshot.

6.4.4 State Diagram:

Fig.6.6 shows the states of one test during execution.

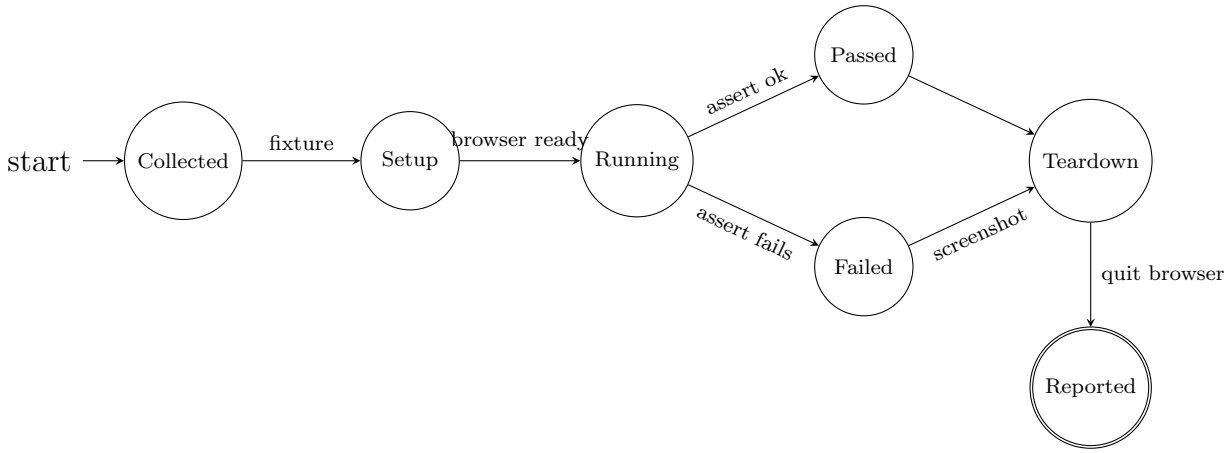


Figure 6.6: State transition diagram of a test

6.4.5 Design Constraints

No Java (Python bindings only); no third-party page-object library; standard pytest plugins only (pytest-html).

6.4.6 Software Interface Description

- **Browser interface:** Selenium WebDriver talks to msedgedriver/chromedriver over the W3C WebDriver protocol; the driver controls the browser.
- **Application interface:** HTTPS pages of Swag Labs.
- **Report interface:** pytest-html writes a self-contained HTML file; screenshots are PNG.

CHAPTER 7

DETAILED DESIGN DOCUMENT USING APPENDIX A AND B

7.1 Introduction

This chapter describes the architecture of the automation framework and its components.

7.2 Architectural Design

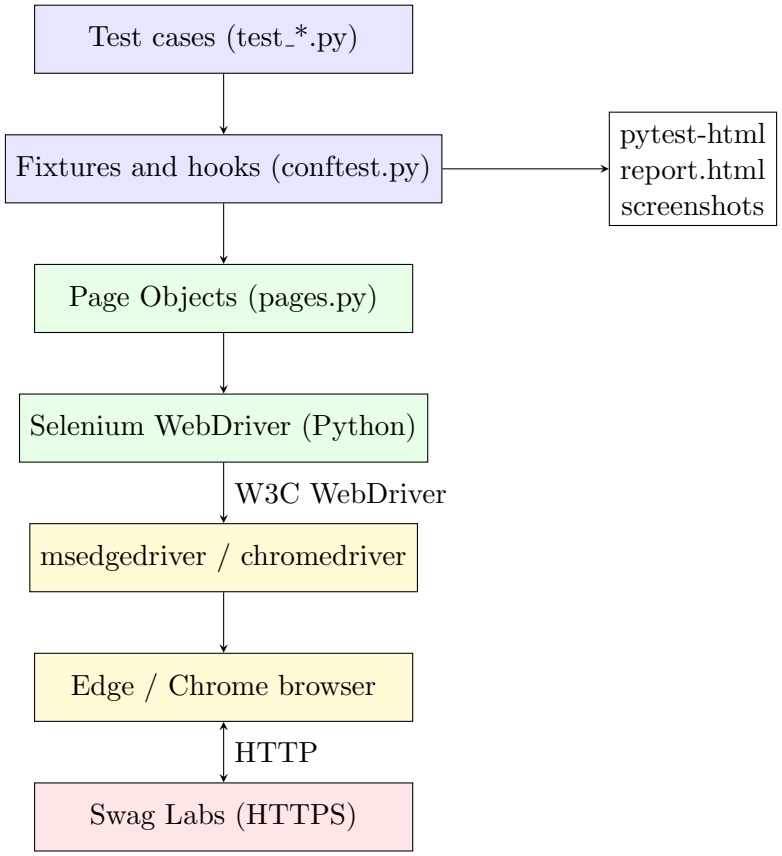


Figure 7.1: Architecture of the automation framework

7.3 Data design (using Appendices A and B)

7.3.1 Internal software data structure

Locator tuples `(By.ID, "login-button")` as class attributes; lists of names and prices returned by page objects; parametrised tuples of test data.

7.3.2 Global data structure

`BASE_URL` and `DEFAULT_PASSWORD` in `pages.py`; `SCREENSHOT_DIR` in `conftest.py`; command-line options `--browser`, `--headed`.

7.3.3 Temporary data structure

The WebDriver session (one per test); pytest's report objects to which screenshots are attached as extras.

7.3.4 Database description

No database. Outputs are files: `reports/report.html`, `reports/console_output.txt`, `reports/screenshots/*.png`.

7.4 Component Design

7.4.1 Class Diagram

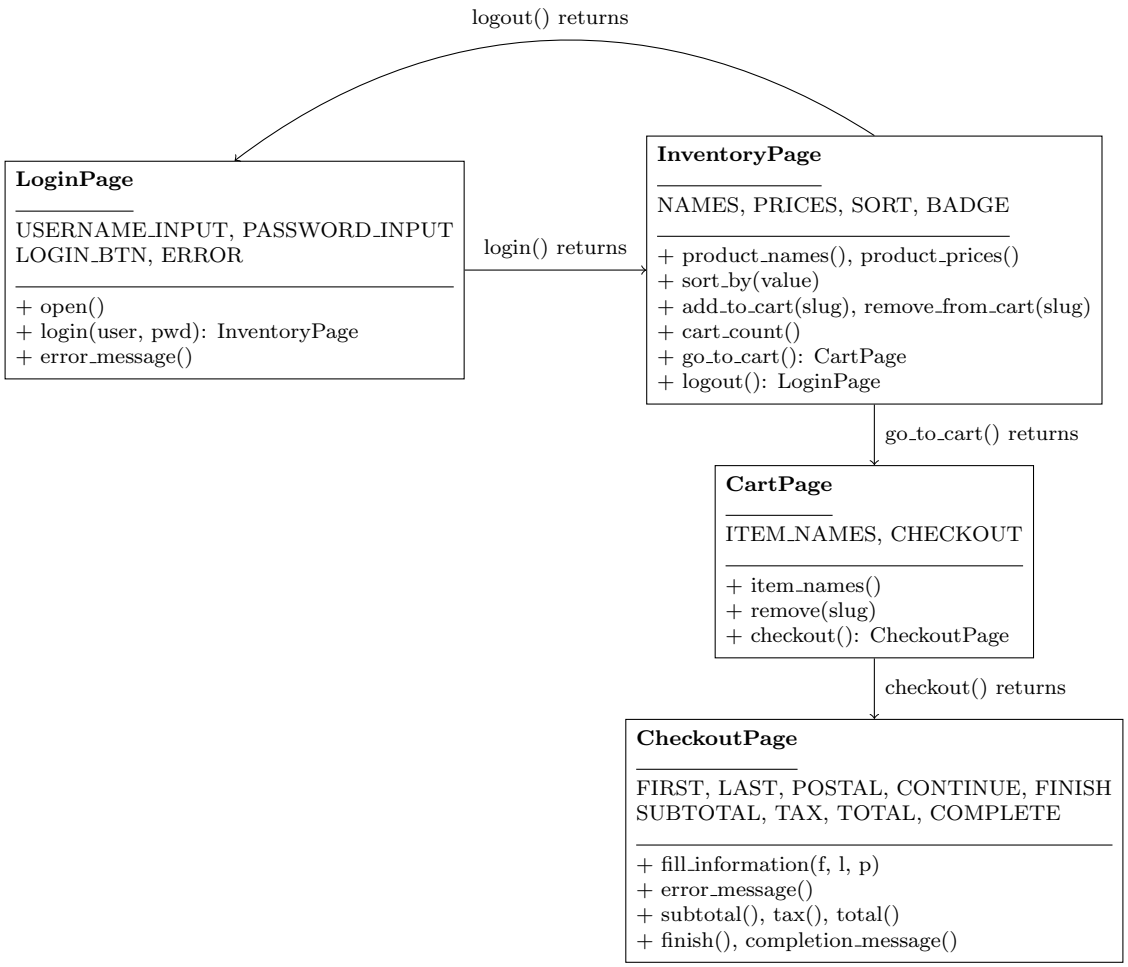


Figure 7.2: Class diagram of the page objects

CHAPTER 8
PROJECT IMPLEMENTATION

8.1 Introduction

The suite consists of `pages.py` (about 140 lines), `conftest.py` (about 60 lines) and five test files (about 120 lines in total). Repository: `https://github.com/VishwajeetTulse/ecommerce-automation-testing`.

8.2 Tools and Technologies Used

- Python 3.14; Selenium 4.51 (WebDriver, Selenium Manager for drivers)
- pytest 9 (fixtures, parametrize, xfail, hooks); pytest-html 4 (report)
- Microsoft Edge 142 headless; Chrome optional
- Visual Studio Code; Git and GitHub

8.3 Methodologies/Algorithm Details

8.3.1 Algorithm 1/Pseudo Code

Algorithm 1: Driver fixture with screenshot on failure

Input: test function t, options browser, headed
Output: verdict, optional screenshot
create browser options; add `--headless=new` unless headed; set window 1280x900;
driver $\leftarrow$ new Edge or Chrome session; implicit wait 5 s;
run t with *driver*;
if t *failed in its call phase* **then**
  save `reports/screenshots/<test name>.png`; attach to HTML report;
quit *driver*;

8.3.2 Algorithm 2/Pseudo Code

Algorithm 2: Checkout total verification test

Input: standard user, products Backpack (29.99) and Bike Light (9.99)
Output: pass if totals are consistent
login; add both products; open cart; click Checkout;
fill first name, last name, postal code; click Continue;
$s \leftarrow$ subtotal shown; $x \leftarrow$ tax shown; $T \leftarrow$ total shown;
assert $s = 39.98$;
assert $x = \text{round}(0.08 \cdot s, 2)$;
assert $T = s + x$;

Key code: the login action with an explicit wait, and a parametrised negative test.

```
def login(self, username, password=DEFAULT_PASSWORD):
    self.driver.find_element(*self.USERNAME_INPUT).send_keys(username)
    self.driver.find_element(*self.PASSWORD_INPUT).send_keys(password)
```

```
    self.driver.find_element(*self.LOGIN_BTN).click()
    # wait until either the Products page opens or an error is shown
    WebDriverWait(self.driver, 10).until(
        lambda d: "inventory.html" in d.current_url or d.find_elements(*self.ERROR))
    return InventoryPage(self.driver)

@pytest.mark.parametrize("username,password,expected", [
    ("", "", "Username is required"),
    ("standard_user", "", "Password is required"),
    ("standard_user", "wrong", "do not match any user"),
    ("locked_out_user", "secret_sauce", "locked out"),
])
def test_invalid_login_shows_error(login_page, username, password, expected):
    login_page.login(username, password)
    assert expected in login_page.error_message()
```

8.4 Verification and Validation for Acceptance

Acceptance criteria: the suite runs unattended with one command; every selected manual case has an automated equivalent; all tests pass on the current site except the documented expected failure; a failed test produces a screenshot; the HTML report is generated. All criteria were verified in the final run (Chapter 9) and by forcing the expected-failure test to run as a normal test, which produced the screenshot shown in Fig.10.2.

CHAPTER 9
SOFTWARE TESTING

9.1 Type of Testing Used

- **Automated functional testing** of each module through the browser (black-box, end-to-end).
- **Regression testing:** the full suite is re-run after any change to the tests or the site.
- **Data-driven testing:** parametrised login and checkout validation cases.
- **Testing of the framework itself:** a deliberately failing run confirmed the screenshot hook and report embedding.

9.2 Test Cases and Test Results

Test	What is verified	Result
test_valid_login	standard_user reaches Products page	Pass
test_invalid_login_shows_error [4 data sets]	empty, no password, wrong password, locked-out user show the right error	Pass (4)
test_password_is_masked	password input type is "password"	Pass
test_six_products_listed	six product names on Products page	Pass
test_sort_price_low_to_high	prices ascending after sort	Pass
test_sort_price_high_to_low	prices descending after sort	Pass
test_sort_name_z_to_a	names in reverse alphabetical order	Pass
test_sort_price_problem_user	sort for problem_user (known defect)	XFAIL
test_add_two_items_updates_badge	badge shows 2	Pass
test_cart_page_lists_added_items	both items listed on cart page	Pass
test_remove_item_from_cart_page	one item left, badge 1	Pass
test_remove_from_products_page	badge disappears	Pass
test_missing_information_shows_error [3 data sets]	first name, last name, postal code required	Pass (3)
test_overview_totals_are_correct	subtotal 39.98, tax 8%, total = sum	Pass
test_complete_order	"Thank you for your order!" page	Pass
test_logout_returns_to_login_page	login page after logout	Pass
test_inventory_page_requires_login	direct URL shows access error	Pass

Table 9.1: Automated test cases and results (final run: 21 passed, 1 xfailed, 105.6 s)

Console summary of the final run:

```
tests/test_cart.py::test_add_two_items_updates_badge PASSED              [  4%]
```

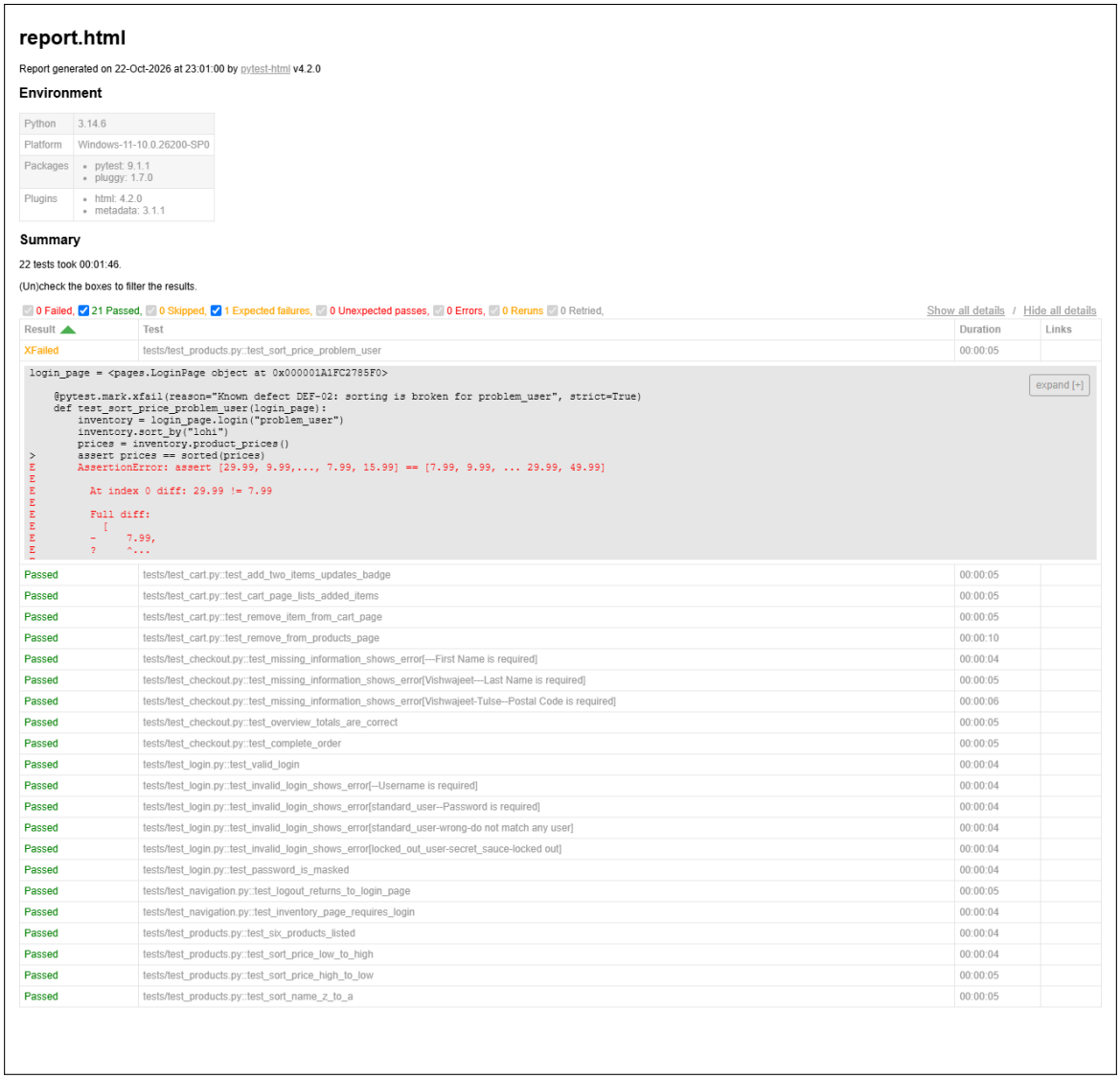
```
tests/test_cart.py::test_cart_page_lists_added_items PASSED              [  9%]
tests/test_cart.py::test_remove_item_from_cart_page PASSED               [ 13%]
tests/test_cart.py::test_remove_from_products_page PASSED                [ 18%]
tests/test_checkout.py::test_missing_information_shows_error[---First Name is required
    ] PASSED
tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet---Last Name is
     required] PASSED
tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet-Tulse--Postal
    Code is required] PASSED
tests/test_checkout.py::test_overview_totals_are_correct PASSED          [ 36%]
tests/test_checkout.py::test_complete_order PASSED                       [ 40%]
tests/test_login.py::test_valid_login PASSED                             [ 45%]
tests/test_login.py::test_invalid_login_shows_error[--Username is required] PASSED
tests/test_login.py::test_invalid_login_shows_error[standard_user--Password is
    required] PASSED
tests/test_login.py::test_invalid_login_shows_error[standard_user-wrong-do not match
    any user] PASSED
tests/test_login.py::test_invalid_login_shows_error[locked_out_user-secret_sauce-
    locked out] PASSED
tests/test_login.py::test_password_is_masked PASSED                      [ 68%]
tests/test_navigation.py::test_logout_returns_to_login_page PASSED       [ 72%]
tests/test_navigation.py::test_inventory_page_requires_login PASSED      [ 77%]
tests/test_products.py::test_six_products_listed PASSED                  [ 81%]
tests/test_products.py::test_sort_price_low_to_high PASSED               [ 86%]
tests/test_products.py::test_sort_price_high_to_low PASSED               [ 90%]
tests/test_products.py::test_sort_name_z_to_a PASSED                     [ 95%]
tests/test_products.py::test_sort_price_problem_user XFAIL (Known de...) [100%]
================== 21 passed, 1 xfailed in 105.64s (0:01:45) ==================
```

A defect found while stabilising the suite is worth recording: in the first run every login failed with "Username and password do not match". The cause was in the page object: the default argument `password=PASSWORD` was evaluated inside the class body, where `PASSWORD` was the locator tuple `(By.ID, "password")`, so Selenium typed the text "idpassword". Renaming the locator to `PASSWORD_INPUT` and the constant to `DEFAULT_PASSWORD` fixed it. The screenshot-on-failure hook made this diagnosis immediate.

CHAPTER 10

RESULTS

10.1 Screen shots



report.html

Report generated on 22-Oct-2026 at 23:01:00 by pytest-html v4.2.0

Environment

Python	3.14.6
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.7.0
Plugins	• html: 4.2.0 • metadata: 3.1.1

Summary

22 tests took 00:01:46.

(Un)check the boxes to filter the results.

0 Failed, 21 Passed, 0 Skipped, 1 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
XFailed	tests/test_products.py::test_sort_price_problem_user	00:00:05	

```
login_page = <pages.LoginPage object at 0x000001A1FC2785F0>

    @pytest.mark.xfail(reason="Known defect DEF-02: sorting is broken for problem_user", strict=True)
    def test_sort_price_problem_user(login_page):
        inventory = login_page.login("problem_user")
        inventory.sort_by("lohi")
        prices = inventory.product_prices()
>       assert prices == sorted(prices)
E       AssertionError: assert [29.99, 9.99,..., 7.99, 15.99] == [7.99, 9.99, ... 29.99, 49.99]
E
E         At index 0 diff: 29.99 != 7.99
E
E         Full diff:
E           [
E         -     7.99,
E         ?     ^...
```

expand [+]

Passed	tests/test_cart.py::test_add_two_items_updates_badge	00:00:05	
Passed	tests/test_cart.py::test_cart_page_lists_added_items	00:00:05	
Passed	tests/test_cart.py::test_remove_item_from_cart_page	00:00:05	
Passed	tests/test_cart.py::test_remove_from_products_page	00:00:10	
Passed	tests/test_checkout.py::test_missing_information_shows_error[—First Name is required]	00:00:04	
Passed	tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet—Last Name is required]	00:00:05	
Passed	tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet-Tulse–Postal Code is required]	00:00:06	
Passed	tests/test_checkout.py::test_overview_totals_are_correct	00:00:05	
Passed	tests/test_checkout.py::test_complete_order	00:00:05	
Passed	tests/test_login.py::test_valid_login	00:00:04	
Passed	tests/test_login.py::test_invalid_login_shows_error[—Username is required]	00:00:04	
Passed	tests/test_login.py::test_invalid_login_shows_error[standard_user–Password is required]	00:00:04	
Passed	tests/test_login.py::test_invalid_login_shows_error[standard_user-wrong-do not match any user]	00:00:04	
Passed	tests/test_login.py::test_invalid_login_shows_error[locked_out_user-secret_sauce-locked out]	00:00:04	
Passed	tests/test_login.py::test_password_is_masked	00:00:04	
Passed	tests/test_navigation.py::test_logout_returns_to_login_page	00:00:05	
Passed	tests/test_navigation.py::test_inventory_page_requires_login	00:00:04	
Passed	tests/test_products.py::test_six_products_listed	00:00:04	
Passed	tests/test_products.py::test_sort_price_low_to_high	00:00:04	
Passed	tests/test_products.py::test_sort_price_high_to_low	00:00:05	
Passed	tests/test_products.py::test_sort_name_z_to_a	00:00:05	

Figure 10.1: pytest-html report of the final run

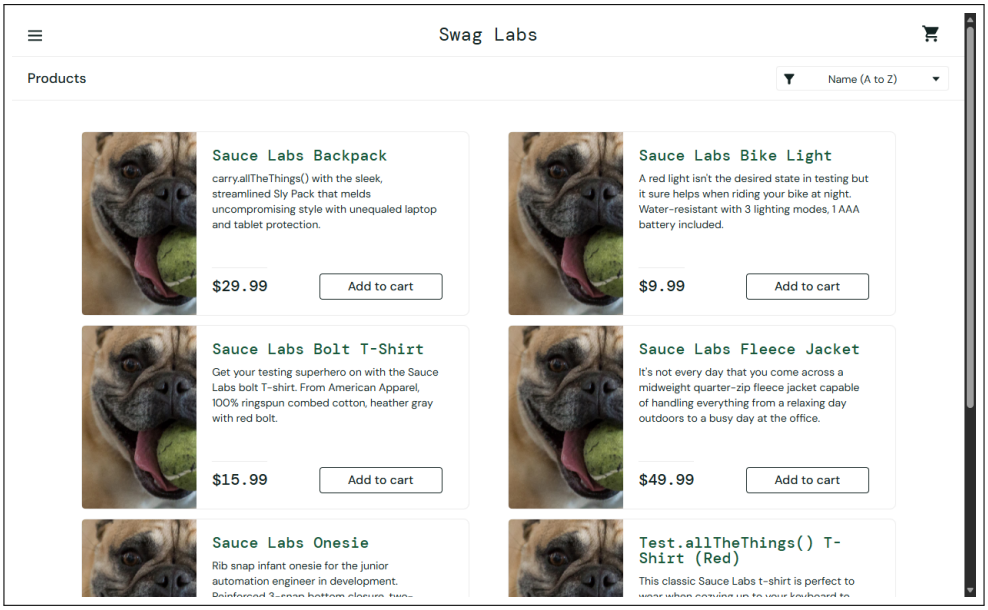


Figure 10.2: Screenshot captured automatically when the expected-failure test is forced to run: prices are not sorted for problem_user

10.2 Outputs

- `reports/report.html`: self-contained HTML report (21 passed, 1 xfailed).
- `reports/console_output.txt`: console log of the run.
- `reports/screenshots/`: PNG captured for each failed test.
- Execution time: 105.6 s for 22 tests in headless Edge, versus about two hours for the same cases executed manually.

CHAPTER 11
DEPLOYMENT AND MAINTENANCE

11.1 Installation and un-installation

```
git clone https://github.com/VishwajeetTulse/ecommerce-automation-testing
cd ecommerce-automation-testing
pip install -r requirements.txt        # selenium, pytest, pytest-html
python -m pytest                       # run everything, headless Edge
```

Selenium Manager downloads the matching browser driver automatically. To un-install, delete the folder and `pip uninstall selenium pytest pytest-html`.

11.2 User help

```
python -m pytest --headed                 # watch the browser
python -m pytest --browser chrome         # use Chrome
python -m pytest tests/test_checkout.py   # one module
python -m pytest -k sort                  # tests whose name contains "sort"
```

Open `reports/report.html` after the run. If a test fails, look at its screenshot in `reports/screenshots/`; if the site's DOM changed, update the locator in `tests/pages.py` only.

CHAPTER 12
CONCLUSION AND FUTURE SCOPE

This mini project automated the regression test cases of an e-commerce website with Selenium WebDriver, Python and pytest. A small Page Object Model keeps locators out of the tests, fixtures give every test an isolated browser, parametrisation covers several data sets with one function, and an HTML report with automatic screenshots makes failures easy to diagnose. The final suite of 22 tests runs unattended in under two minutes and currently passes except for one documented defect kept as an expected failure.

Future scope: run the suite in a CI pipeline (GitHub Actions) on every commit, add cross-browser runs on Selenium Grid, extend coverage to the remaining special users, add API-level checks, and integrate with a test-management tool for traceability to the manual test cases.

CHAPTER 13
REFERENCES

[1] V. Garousi and M. V. Mäntylä. 2016. When and what to automate in software testing? A multi-vocal literature review. *Information and Software Technology* 76 (2016), 92–117.

[2] M. Leotta, D. Clerissi, F. Ricca and C. Spadaro. 2013. Improving test suites maintainability with the Page Object pattern: an industrial case study. In *Proc. ICST Workshops*. IEEE, 108–113.

[3] M. Leotta, D. Clerissi, F. Ricca and P. Tonella. 2014. Visual vs. DOM-based web locators: an empirical study. In *Proc. International Conference on Web Engineering (ICWE)*. Springer, 322–340.

[4] A. Bruns, A. Kornstadt and D. Wichmann. 2009. Web application tests with Selenium. *IEEE Software* 26, 5 (2009), 88–91.

[5] P. Ramya, V. Sindhura and P. V. Sagar. 2017. Testing using Selenium web driver. In *Proc. 2nd International Conference on Electrical, Computer and Communication Technologies (ICECCT)*. IEEE, 1–7.

[6] D. M. Rafi, K. R. K. Moses, K. Petersen and M. V. Mäntylä. 2012. Benefits and limitations of automated software testing: systematic literature review and practitioner survey. In *Proc. 7th International Workshop on Automation of Software Test (AST)*. IEEE, 36–42.

[7] S. Berner, R. Weber and R. K. Keller. 2005. Observations and lessons learned from automated testing. In *Proc. 27th International Conference on Software Engineering (ICSE)*. ACM, 571–579.

[8] Selenium Project. *Selenium WebDriver Documentation*. `https://www.selenium.dev/documentation/webdriver/`

[9] pytest Developers. *pytest Documentation*. `https://docs.pytest.org/`

[10] Sauce Labs. *Swag Labs sample application*. `https://www.saucedemo.com`

[11] R. S. Pressman. 2010. *Software Engineering: A Practitioner's Approach* (7th ed.). McGraw-Hill.

ANNEXURE A

LABORATORY ASSIGNMENTS ON PROJECT ANALYSIS OF ALGORITHMIC DESIGN

- **IDEA Matrix.** Increase: regression frequency. Drive: fast feedback to developers. Educate: students on WebDriver and pytest. Accelerate: a two-hour manual run to two minutes. Improve: diagnosability through screenshots. Deliver: one-command suite and report. Evaluate: pass/fail per module. Associate: automated tests with manual case IDs. Ignore: visual pixel checks. Decrease: duplicated locators. Eliminate: fixed sleeps. Avoid: inter-test dependencies.
- **Feasibility.** Execution is $O(n)$ in the number of tests; each test performs a constant number of browser actions. No NP-hard component.
- **Function.** $x =$ (test, data set), $y = f(x) \in \{\text{pass}, \text{fail}, \text{xfail}\}$, with f evaluated by executing the test's assertions against the live application.

ANNEXURE B

LABORATORY ASSIGNMENTS ON PROJECT QUALITY AND RELIABILITY TESTING OF PROJECT DESIGN

- **Objects and morphisms.** Objects: WebDriver, LoginPage, InventoryPage, CartPage, CheckoutPage, Test, Report. Morphisms: open (driver → LoginPage), login (LoginPage → InventoryPage), go_to_cart (InventoryPage → CartPage), checkout (CartPage → CheckoutPage), execute (Test → verdict). The page-object methods compose: checkout ∘ go_to_cart ∘ login ∘ open.

- **Functional dependency graph.** Tests depend on fixtures (`driver` → `login_page` → `inventory` → `checkout`); fixtures depend on page objects; page objects depend on Selenium. See Fig.7.1.

- **Test data.** Parametrised sets for login (4) and checkout validation (3); product slugs `sauce-labs-backpack`, `sauce-labs-bike-light`, `sauce-labs-onesie`; expected amounts 39.98 / 3.20 / 43.18.

- **Black-box test cases.** Listed in Table 9.1; each corresponds to a manual test case of the companion project.

ANNEXURE C
PROJECT PLANNER

Week	Activity	Deliverable	Status
1	Install Python, Selenium, pytest; first script; select manual cases to automate	Environment, case list	Done
2	Page objects, driver fixture, login and products tests	`pages.py`, `conftest.py`, 2 test files	Done
3	Cart, checkout, navigation tests; xfail; HTML report; screenshot hook	5 test files, `pytest.ini`	Done
4	Full runs, fix login default-argument defect, README, repository, report	report.html, report.pdf	Done

Table C.1: Project planner

ANNEXURE D

REVIEWERS COMMENTS OF PAPER SUBMITTED

Not applicable for this mini project.

ANNEXURE E
PLAGIARISM REPORT

To be attached after checking through the institute's plagiarism tool.

ANNEXURE F

TERM-II PROJECT LABORATORY ASSIGNMENTS

1. Review of design: the guide's suggestion to keep page objects minimal and avoid a heavy framework was followed.

2. Project workstation: Windows 11 laptop with Python 3.14, Selenium 4.51, pytest 9, Microsoft Edge 142, VS Code and Git; Selenium Manager handled the driver.

3. Programming: completed as described in Chapter 8.

4. Testing: final run 21 passed, 1 xfailed in 105.6 s; report and screenshots in Chapter 10.

ANNEXURE G

INFORMATION OF PROJECT GROUP MEMBERS

1. Name : Vishwajeet Tulse
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 2
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 3
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 4
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –